

Contents

Setup — Für den Dozenten	1
Voraussetzungen	1
Projekt öffnen	1
App starten	1
Tests ausführen	1
Wenn etwas nicht kompiliert	2
Vor dem Praktikum: Basis-Branch anlegen	2
Integration am Ende jedes Tages	2
Bekannte Stolpersteine	2

Setup — Für den Dozenten

Diese Anleitung geht davon aus, dass IntelliJ IDEA bereits an Tag 1/2 installiert und getestet wurde.

Voraussetzungen

- IntelliJ IDEA (Community reicht) mit Kotlin-Plugin (ist standardmäßig aktiv)
- JDK 17 oder neuer (die JetBrains Runtime, die mit IntelliJ mitgeliefert wird, reicht als Basis — für den Gradle-Build selbst lädt sich das Projekt bei Bedarf automatisch ein JDK 21 nach, siehe unten)
- Internetzugang beim ersten Öffnen des Projekts (Gradle lädt Compose- und Kotlin-Dependencies von Maven Central und dem JetBrains-Repo herunter, bei Bedarf zusätzlich ein JDK 21 über den Foojay-Dienst, danach liegt alles im lokalen Gradle-Cache und funktioniert auch offline)

Projekt öffnen

1. IntelliJ starten -> **Open** -> den Ordner bot-match auswählen (nicht eine einzelne Datei).
2. IntelliJ erkennt automatisch das Gradle-Projekt und importiert es. Das dauert beim ersten Mal 2-5 Minuten (Dependency-Download). Unten rechts läuft ein Fortschrittsbalken ("Sync" / "Indexing").
3. Falls IntelliJ nach einem JDK fragt: die vorgeschlagene JetBrains-Runtime (JBR) auswählen, oder falls keine automatisch erkannt wird, unter **File -> Project Structure -> SDK** ein JDK 17+ setzen.

App starten

Option A (empfohlen für Schüler): Datei `src/main/kotlin/framework/Main.kt` öffnen, auf den grünen Play-Button links neben `fun main()` klicken.

Option B (Terminal):

```
./gradlew run
```

Die App öffnet ein Fenster mit der 10×10-Arena links, Steuerung/Scoreboard/Log rechts. Standardmäßig sind alle Bots aus BotRegistry (Beispiel-Bots + aktueller Stand aller drei Teams) als Kandidaten in der Checkbox-Liste auswählbar.

Tests ausführen

```
./gradlew test
```

Führt die Engine-Unit-Tests aus (`GameEngineTest.kt`). Praktisch, um schnell zu prüfen, dass die Kernlogik nach eigenen Anpassungen noch funktioniert — für Schüler-Bot-Code selbst sind keine Pflicht-Tests vorgesehen (siehe optionale Aufgabe in Epic 4 im Backlog).

Wenn etwas nicht kompiliert

Der häufigste Fehler bei Schülern: eine Bot-Datei hat einen Syntaxfehler oder die neue Bot-Klasse wurde nicht in die `teamXBots`-Liste am Ende der Datei eingetragen. Beides zeigt IntelliJ sofort als roten Unterstrich/Fehlermeldung in der Datei an, bevor überhaupt gebaut wird — das ist gewollt (frühes, sichtbares Feedback statt stiller Laufzeitfehler).

Falls Gradle nach einem Fehler “hängt” oder komisch reagiert: **File -> Invalidate Caches -> Invalidate and Restart**, danach nochmal Gradle-Sync abwarten.

Vor dem Praktikum: Basis-Branch anlegen

Für jeden Praktikums-Durchlauf legst du vorab einen eigenen Basis-Branch an, z.B. `student-2026_07` (Namensschema: Jahr_Monat des Termins). Jedes Team zweigt zu Beginn seinen eigenen Branch davon ab (`student-2026_07-A/-B/-C`) und pusht darauf täglich seinen Fortschritt. Details zum Ablauf (Branch erstellen, committen, Pull Request) stehen in [docs/git-github-basics.md](#) — die ist auch für die Schüler gedacht.

Integration am Ende jedes Tages

Statt eines manuellen Zusammenführens am Turniertag läuft die Integration über Git: Am Ende jedes Tages öffnet jedes Team einen Pull Request von seinem Team-Branch zurück auf den Basis-Branch (z.B. `student-2026_07-A -> student-2026_07`) — das übernimmst am besten du, damit alle drei Teams zuverlässig und zur gleichen Zeit gemergt werden. Danach:

1. Sicherstellen, dass jede Team-Datei `bots/teamX/TeamXBots.kt` nach dem Merge noch kompiliert (`./gradlew build` auf dem Basis-Branch laufen lassen, damit vor dem Testduell keine Überraschungen warten).
2. `BotRegistry.kt` muss nicht verändert werden, sofern jedes Team seine Bots brav in die vorgesehene `teamXBots`-Liste einträgt — die Registry liest diese automatisch ein.
3. App starten (auf dem Basis-Branch), alle gewünschten Bots in der Checkbox-Liste auswählen, Battle-Royale laufen lassen.

Das Gleiche passiert am Ende von Tag 3 noch einmal für das Turnier-Finale.

Bekannte Stolpersteine

- **Firmen-/Schulnetz blockt Maven Central:** Vor dem Praktikumstag einmal selbst `./gradlew build` im Zielnetz testen. Falls es klemmt, das Projekt vorher in einem funktionierenden Netz einmal komplett bauen (füllt den lokalen Gradle-Cache unter `~/.gradle`) und diesen Cache auf die Praktikumsrechner mitbringen.
- **Nur JDK 17 auf dem Rechner, kein JDK 21:** Das Projekt braucht für den Build JDK 21 (`jvmToolchain(21)` in `build.gradle.kts`). `settings.gradle.kts` bindet dafür den Foojay-Resolver ein, der bei Bedarf automatisch ein passendes JDK 21 herunterlädt — das passiert beim ersten Build mit, solange Internetzugang besteht. Wird der Gradle-Cache vorab in einem funktionierenden Netz gefüllt (siehe Punkt oben), ist das heruntergeladene JDK darin enthalten und der Praktikumsrechner braucht selbst keinen Internetzugang mehr dafür.
- **Mehrere Bot-Klassen pro Team:** kein Problem, einfach weitere Klassen in derselben Datei (oder neue Dateien im selben Package) anlegen und alle in die `teamXBots`-Liste aufnehmen.
- **App reagiert nicht mehr / ein Bot scheint zu hängen:** das Framework fängt Endlosschleifen in Schüler-Code automatisch ab (nach 3 Timeouts wird der betroffene Bot für den Rest des

Matches auf “Wait” gesetzt, sichtbar im Log). Die App selbst friert dabei nicht ein — falls doch, ist es ein echter Bug im Framework, nicht im Schülercode.